Session 5 — Git in the IDE

Shared topic (Python FS + Java FS) · **~60–75 minutes** · homework is fine

Index: Git and GitHub
Previous: Session 4 — Sync, undo, and conflicts
Next (Python FS): MySQL for Full Stack Developers
Next (Java FS): MySQL for Full Stack Developers

Session Outcome

No new Git concepts. The buttons run the same commands you already used in the terminal.

By the end you will be able to clone, stage, commit, branch, pull, push, merge, resolve a conflict, and open a Pull Request from the editor — and still check the truth with `git status`.

Course default IDE is **VS Code** (both programs). Then use the map for **PyCharm** (Python) or **IntelliJ** (Java) if that is the editor you live in.

If a button is confusing, run the equivalent command in the terminal. The terminal wins.

1. Why the IDE is last

The IDE is faster once the packing line is in your head.

If you start with buttons, **Commit All** and **Sync** hide staging and hide pull-before-push. You click, GitHub changes, you cannot say why.

Same picture as Session 2:

```
working folder  →  staging  →  commit  →  push  →  GitHub
```

The IDE is another interface for that picture. It is not a second Git.

2. VS Code (everyone)

Open `~/KSR_MART` with **File → Open Folder**. Do not open the Desktop (parent of many folders).

Source Control: the branch icon in the left bar, or `Ctrl+Shift+G` / `Cmd+Shift+G`.

Look at the **status bar** (bottom left). The branch name should say `main` or `feature/login`. That is `git branch` with a star.

You already know	In VS Code
Working folder	Explorer + **Changes** (`U` untracked, `M` modified)
`git add FILE`	**+** on that file (Stage)
`git add .`	**Stage All** — same risk
`git commit -m`	message box + **Commit**
`git status` / `git diff`	the list; click a file for the diff

You already know	In VS Code
`git restore --staged`	**Unstage** (–)
`git restore FILE`	**Discard Changes** — edits can be gone
`git switch -c`	status-bar branch → **Create new branch**
`git switch`	status-bar branch list
`git pull` / `git push`	⋯ menu, or **Sync** (pull then push)
`git log`	**Timeline** on a file, or **Git: View History** in the Command Palette
`git clone`	**Clone Repository** on the Welcome screen, or Command Palette **Git: Clone**
Pull Request	GitHub in the browser (Session 3), or the **GitHub Pull Requests** extension if installed
Conflict	merge editor: **Current** / **Incoming** / **Accept Both**, then stage

Clone asks for the GitHub URL and a parent folder. It is `git clone`. Do not `git init` in that new folder.

Sync talks to GitHub. It does **not** replace Commit. Uncommitted work (even staged) is not on GitHub until you Commit, then Push or Sync.

If Git is missing: VS Code cannot see `git` on PATH. Install Git, restart, run `git --version` in the VS Code terminal.

Type-along

1. Change `README.md`. Create `scratch.txt`.
2. Stage **only** `README.md`. Leave `scratch.txt` unstaged.
3. Message: `Update README from VS Code`. **Commit**.
4. Status bar: create `feature/ide-demo`. Add one line to `NOTES.txt`. Stage. Commit `Add note from IDE demo`.
5. Switch to `main`. Command Palette: **Git: Merge Branch...** → `feature/ide-demo` (or merge in the terminal).
6. **Pull** if GitHub moved, then **Push** (or Sync on `main` after you looked at the branch name).

Confirm in the terminal:

```
cd ~/KSR_MART
git status
git log --oneline -5
```

The log must match what you clicked. If it does not, the button did something else — believe `status`.

Conflict in the merge editor

Cause a conflict the Session 4 way (two branches, same README line), or open lab `02` broken README.

- **Current** = the branch you are on (`HEAD`)
- **Incoming** = the other side

Accept Current, Incoming, or Both, then **edit** if both are incomplete. Remove leftover `<<<<<<<` markers. **Stage** the file. Complete the merge commit.

Accept Both can duplicate headings. Read the file before you commit.

Pull Request from VS Code

The website from Session 3 is enough: push `feature/login`, then **Compare & pull request** on GitHub.

If you install **GitHub Pull Requests and Issues**, you can create and merge the PR in the editor. Same rules: base `main`, compare the feature branch, do not force-push `main`.

---

3. PyCharm (Python FS)

Skip this section if you are on the Java batch and only use IntelliJ.

You already know	In PyCharm
Source Control list	**Commit** tool window (left, or `Ctrl+K` / `Cmd+K`)
`git add FILE`	checkbox next to that file
`git commit -m`	message + **Commit**
Diff	click the file in Commit
`git switch` / `switch -c`	bottom-right branch name, or **Git** → **New Branch**
`git pull`	**Git** → **Pull**
`git push`	**Git** → **Push**
`git clone`	**Get from VCS** / **Git** → **Clone**
History	**Git** tool window (log)
Merge	**Git** → **Merge**
Conflict	merge dialog: yours / theirs / both, then commit
Pull Request	GitHub in the browser, or the GitHub plugin if you added it

Commit and Push is one button. That is `commit` + `push`. If you meant to stay local, use **Commit** only.

Open `~/KSR_MART` as the project folder, not a parent. Repeat the VS Code type-along with checkboxes instead of `+`. Then `git log --oneline -3` in PyCharm's terminal.

---

4. IntelliJ (Java FS)

Skip this section if you only use VS Code or PyCharm.

IntelliJ Community uses the same packing line as PyCharm. You do not relearn Git.

You already know	In IntelliJ
Stage + commit	**Commit** tool window: checkboxes + message
Push	**Git** → **Push**, or **Commit and Push**

You already know	In IntelliJ
Pull	**Git** → **Pull** (or Update Project)
Branch	status bar, **Git** → **New Branch**
Merge / conflict	**Git** → **Merge**, then the merge dialog
Clone	**Get from VCS**
History	**Git** tool window

Find three things: the packing list (checkboxes), the message, the branch name. The rest is Session 2–4.

5. Try this

In **one** IDE, on `~/KSR_MART`:

1. Stage one file, not all files. Commit with an imperative message.
2. Create `feature/ide-session5`, commit one small change, push the branch.
3. Open a Pull Request on GitHub (browser is fine). Merge it. **Pull** `main` in the IDE.
4. In the terminal: `git log -1` and `git status`. They must match the UI.

No new lab kit. Reuse KSR_MART or labs/01_git_portfolio_hygiene.

Common mistakes

Stage All / Commit All when only two files belong together. Same as `git add .`.

Sync on the wrong branch. Read the status-bar name first.

Discard Changes to "clean the list." That is `git restore FILE`. The typing can be gone.

Sync before Commit. Sync uploads **commits**. Staged-but-uncommitted work stays on this laptop.

Commit and Push when you only wanted a local checkpoint. Use Commit, then Push when you mean it.

Force Push in the IDE menu on `main`. Same rule as Session 4: never.

Key takeaway

```
Explorer / Changes     →  working folder
+  or checkbox         →  git add
message + Commit       →  git commit
Unstage                →  git restore --staged
Discard                →  git restore FILE   (destructive)
status-bar branch      →  git switch
Pull / Push            →  git pull / git push
Sync                   →  pull then push
```

If the UI and `git status` disagree, believe the terminal.

You now have the whole 6-hour course: history on the laptop, a copy on GitHub, a daily loop, a feature branch with a Pull Request, safe desk fixes, and the same moves in the editor.

Work on KSR_MART the way a teammate would: small commits, ignore secrets, pull before you push, keep `main` demo-ready.